Bancos de Dados Não-Relacionais

Aula 04: Modelo Chave-Valor, Arquitetura em Memória e Estruturas Nativas do Redis

A presentation by
Prof. Me. Reginaldo Fernandes @ Instituto Federal do Ceará – Campus Tauá Tecnologia em Análise e Desenvolvimento de Sistemas

INSTITUTO FEDERAL
Ceará
Campus Tauá

13 de Outubro de 2026

Agenda do Encontro

O Modelo Chave-Valor e o Ecossistema Redis

Arquitetura em Memória e Mecanismos de Persistência

Variações Avançadas dos Comandos SET e GET

A Importância do Cache, Expiração e Práticas de Atualização

Estruturas Nativas e Modernização com Inteligência Artificial

O Modelo Chave-Valor e o Ecossistema Redis

O Paradigma Chave-Valor (Key-Value)

Simplicidade e Desempenho Extremo

- Entre os 4 modelos NoSQL, o **Chave-Valor** é o mais primitivo e veloz.
- Cada dado é indexado por uma única chave opaca (*Key*).
- O valor associado (*Value*) é tratado como uma carga útil (*payload*).
- **Complexidade de Busca:** Tempo constante $\mathcal{O}(1)$ via tabela hash interna.

Por que o Redis se Destaca no Mercado?

- Criado por **Salvatore Sanfilippo (antirez)** em 2009.
- Sigla: **REmote DIctionary Server**.
- **In-Memory Engine:** Opera 100% na memória RAM com latência sub-milissegundo (< 1 ms).
- Não armazena apenas strings brutas, mas **estruturas de dados ricas**.

Ambiente Windows: Por que Usar o WSL 2?

Natureza POSIX do Redis

- O Redis depende de primitivas POSIX (Linux/Unix), como `fork()` para persistência em segundo plano.
- Não há binário oficial para Windows nativo desde a versão 3.2.
- Versões não-oficiais para Windows apresentam instabilidade e faltam recursos modernos.

A Solução Definitiva: WSL 2

- O **Windows Subsystem for Linux 2** roda um kernel Linux real sobre hipervisor leve.
- **Desempenho Idêntico ao Linux:** Sem degradação em chamadas de sistema.
- **Rede Transparente:** O Windows mapeia `localhost:6379` automaticamente para IDEs e Python.

Instalação e Execução do Redis no WSL 2

1. Instalação do Ambiente Linux

- No PowerShell (Administrador):

```
# Instala o Ubuntu no WSL 2:
wsl --install -d Ubuntu
```

2. Pacote Oficial no Ubuntu

```
# Atualiza repositorios e instala:
sudo apt update
sudo apt install redis-server -y
```

3. Gerenciamento do Serviço

```
# Iniciar o servidor Redis:
sudo service redis-server start

# Verificar se esta rodando:
sudo service redis-server status
```

4. Teste de Conexão

```
# Abrir console interativo:
redis-cli
127.0.0.1:6379> PING
PONG
```

Convenção de Chaves e Diagnóstico no redis-cli

Convenção de Namespaces

- O Redis não possui schemas relacionais.
- Chaves são organizadas com dois-pontos (`:`):
- **Padrão:** `modulo:entidade:id:campo`
- **Exemplo:** `loja:prod:42:preco`
- Garante legibilidade e isolamento.

Diagnóstico no redis-cli

```
# Testar eco e latencia:
127.0.0.1:6379> PING "IFCE Taua"
"IFCE Taua"

# Diagnostico do servidor:
127.0.0.1:6379> INFO server
# Retorna versao, PID e uptime

# Contagem total de chaves:
127.0.0.1:6379> DBSIZE
(integer) 42
```

Arquitetura em Memória e Mecanismos de Persistência

A Memória RAM: Velocidade Extrema vs Volatilidade

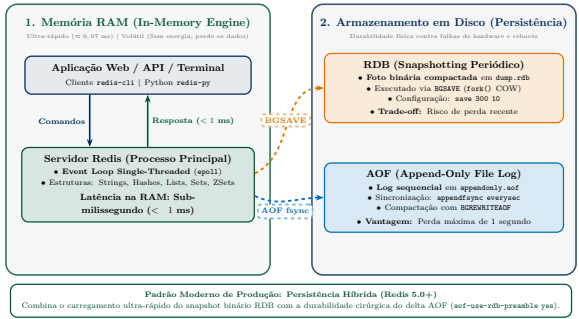
Por que o Redis é Tão Rápido?

- Acesso direto à memória RAM sem overhead de barramento mecânico ou NVMe.
- Latência típica: **70 microssegundos** ($\approx 0,07$ ms).
- **O Dilema Físico:** A memória RAM é **volátil**. Se o servidor reiniciar ou faltar energia, todos os dados são apagados.

A Solução: Persistência em Disco

- O Redis não é apenas um cache efêmero; ele pode operar como banco de dados durável.
- O motor oferece dois mecanismos nativos de armazenamento não-volátil:
- **1. RDB (Snapshotting):** Fotografias pontuais em disco.
- **2. AOF (Append-Only File):** Diário contínuo de operações.

Arquitetura em Memória vs Armazenamento Persistente



Mecanismo 1: RDB (Snapshotting e BGSAVE)

Como Funciona o Snapshotting

- O RDB grava uma cópia binária compactada de toda a memória no arquivo `dump.rdb`.
- Ideal para **backups periódicos** e recuperação de desastres.
- **Ponto de Restauração:** Se houver falha, dados gerados após o último snapshot são perdidos.

Diferença Crítica: BGSAVE vs SAVE

- **SAVE (Síncrono / Bloqueante):** Trava a thread do Redis. Nenhuma consulta é respondida até terminar. **Nunca use em produção!**
- **BGSAVE (Assíncrono / Não-bloqueante):** Dispara processo filho via `fork()` com *Copy-on-Write* (COW). Atende conexões sem interrupção.

Configuração do RDB e Comandos no redis-cli

Regras de Disparo no redis.conf

```
# Sintaxe: save <segundos> <alteracoes>
save 900 1      # 15 min se 1 chave mudou
save 300 10     # 5 min se 10 chaves mudaram
save 60 10000   # 1 min se 10k chaves mudaram

# Destino do snapshot binario
dbfilename dump.rdb
dir /var/lib/redis
```

Disparo Manual via redis-cli

```
# Disparar snapshot assincrono:
127.0.0.1:6379> BGSAVE
Background saving started

# Timestamp UNIX do ultimo dump:
127.0.0.1:6379> LASTSAVE
(integer) 1791888929

# Inspecionar status de persistencia:
127.0.0.1:6379> INFO persistence
# rdb_last_bgsave_status:ok
```

Mecanismo 2: AOF (Append-Only File)

Diário Sequencial de Operações

- Registra cada comando de escrita recebido em modo append (`appendonly.aof`).
- Na reinicialização, o Redis reproduz o diário para reconstruir o estado com quase zero perda.
- **BGREWRITEAOF:** Compacta o diário em background removendo comandos obsoletos.

Políticas de Sincronização (fsync)

- **appendfsync always:** Sincroniza a cada comando. Durabilidade máxima, menor vazão.
- **appendfsync everysec (Padrão Recomendado):** Descarrega em disco a cada 1 segundo. Máxima perda: 1 segundo de dados.
- **appendfsync no:** O SO decide quando gravar.

Configuração do AOF e Persistência Híbrida

Diretivas no redis.conf

```
# Habilitar o modo Append-Only
appendonly yes
appendfilename "appendonly.aof"

# Frequencia de flush fsync em disco
appendfsync everysec

# Persistencia Hibrida (Padrao
     Redis 5+)
# Salva RDB no inicio + AOF no final
aof-use-rdb-preamble yes
```

Compactação e Inspecção no CLI

```
# Reescrever diario AOF em
     background:
127.0.0.1:6379> BGREWRITEAOF
Background append only file
     rewriting started

# Consultar estatisticas detalhadas:
127.0.0.1:6379> INFO persistence
# aof_enabled:1
# aof_rewrite_in_progress:0
```

Variações Avançadas dos Comandos SET e GET

Problema Real Inicial: Cache de Termos de Busca

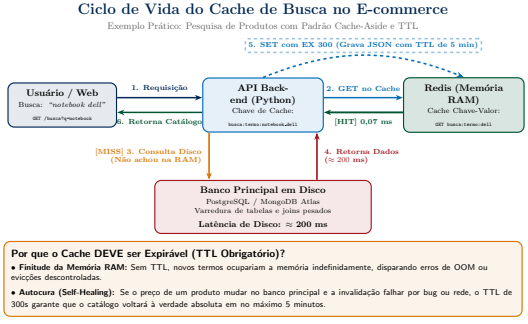
O Gargalo de Consultas Textuais

- No e-commerce da Tauá Tech Store, milhares de clientes pesquisam simultaneamente por “*notebook dell*”.
- Realizar buscas em bancos relacionais com `LIKE '%notebook%'` ou regex no MongoDB exige varredura pesada e leva de 150 a 250 ms.
- **Modelagem Chave-Valor:**
- **Chave:** `busca:termo:notebook_dell`
- **Valor:** JSON com lista de produtos encontrados.

A Estratégia de Aceleração

- Armazena o resultado em memória no Redis com TTL de 300 segundos (**5 minutos**).
- **Cache Hit:** A resposta é devolvida em **0,07 ms** ($\approx 2800\times$ mais rápido!).
- **Cache Miss:** Primeira consulta calcula no banco e alimenta o Redis.
- Contadores atômicos com `INCR` medem a popularidade de cada termo em tempo real.

Ciclo de Vida do Cache de Busca no E-commerce



SET com Expiração Temporal (EX e PX)

Expiração Temporal (TTL)

- O parâmetro `EX` define a expiração em segundos inteiros.
- O parâmetro `PX` define a expiração em milissegundos.
- Evita acúmulo descontrolado de dados obsoletos na memória RAM.

Comandos no redis-cli

```
# Gravar catalogo com TTL de 300s (5
    min):
127.0.0.1:6379> SET busca:dell
    "[{id:1}]" EX 300
OK

# Consultar TTL restante em segundos:
127.0.0.1:6379> TTL busca:dell
(integer) 298

# Gravar chave com PX (2500 ms):
127.0.0.1:6379> SET ping:host
    "10.0.0.1" PX 2500
OK
```

SET com Trava Distribuída (Lock NX)

Exclusão Mútua Atômica

- O modificador `NX` grava **apenas se a chave não existir**.
- Se a chave já existir, a gravação falha e devolve `(nil)`.
- **Caso Real:** Trava de concorrência para reservar a última unidade de um estoque.

Comandos no redis-cli

```
# Worker A adquire a trava por 15s:
127.0.0.1:6379> SET lock:prod:101
     "wA" NX EX 15
OK

# Worker B tenta simultaneamente:
127.0.0.1:6379> SET lock:prod:101
     "wB" NX EX 15
(nil)  # Bloqueado com sucesso!

# Liberacao manual da trava:
127.0.0.1:6379> DEL lock:prod:101
(integer) 1
```

SET Condicional (XX) e Preservação de TTL (KEEPTTL)

Atualização Estrita e TTL

- `XX`: Grava **apenas se a chave já existir** (útil para atualizar status sem criar lixo).
- `KEEPTTL`: Atualiza o conteúdo da chave preservando a contagem de tempo restante.

Comandos no redis-cli

```
# Tentar atualizar chave inexistente:
127.0.0.1:6379> SET status:servico
     "on" XX
(nil)  # Nao cria chave nova

# Criar sessao com TTL de 60s:
127.0.0.1:6379> SET sessao:u1
     "token_v1" EX 60
OK

# Atualizar valor mantendo o tempo:
127.0.0.1:6379> SET sessao:u1
     "token_v2" KEEPTTL
OK  # O relogio regressivo nao zera!
```

SET com Troca Atômica (GET) e Lotes (MSET)

Substituição e Multi-Escrita

- `SET ... GET`: Atualiza o valor e devolve o valor anterior atomicamente.
- `MSET`: Grava múltiplas chaves e valores em um único pacote TCP.
- Evita múltiplas viagens de ida e volta (RTT).

Comandos no redis-cli

```
# Substituir valor e receber o
    antigo:
127.0.0.1:6379> SET contagem:vendas
    100 GET
"85"  # Retornou o valor anterior!

# Gravar varias moedas em 1 RTT:
127.0.0.1:6379> MSET usd 5.45 eur
    5.95 btc 345k
OK

# Leitura conjunta imediata:
127.0.0.1:6379> MGET usd eur
1) "5.45"  2) "5.95"
```

GET Simples e em Lote (MGET)

Recuperação de Alta Velocidade

- `GET`: Recupera uma única chave em $\mathcal{O}(1)$.
- `MGET`: Recupera múltiplas chaves de forma atômica.
- Chaves inexistentes retornam `(nil)` sem interromper a consulta das demais.

Comandos no redis-cli

```
# Leitura simples de termo cacheado:
127.0.0.1:6379> GET busca:dell
"[{id:1}]"

# Consulta de multiplas chaves:
127.0.0.1:6379> MGET usd eur
     chave_inexistente
1) "5.45"
2) "5.95"
3) (nil)
```

GETDEL: Consumo Atômico de Tokens de Uso Único

Leitura e Destruição em $\mathcal{O}(1)$

- Lê o valor e **remove a chave atomicamente** em uma única operação.
- Elimina vulnerabilidade de reutilização concorrente (*double spending*).
- **Casos Reais:** Cobrança PIX, tokens de redefinição de senha e ingressos.

Comandos no redis-cli

```
# Gerar token PIX de 60 segundos:
127.0.0.1:6379> SET token:pix:99
     "R$500" EX 60
OK

# Consumir e apagar atomicamente:
127.0.0.1:6379> GETDEL token:pix:99
"R$500"

# Segunda tentativa de leitura:
127.0.0.1:6379> GET token:pix:99
(nil)  # Ja consumido e destruido!
```

GETEX: Renovação Deslizante de Sessão

Extensão de Vida na Leitura

- `GETEX`: Lê o dado e atualiza o seu TTL no mesmo instante.
- **Sessão Deslizante:** Enquanto o usuário navega na aplicação, a expiração é postergada sem comandos adicionais de escrita.

Comandos no redis-cli

```
# Sessao com TTL de 30s:
127.0.0.1:6379> SET sessao:u42
     "jwt_token" EX 30
OK

# Ler e estender para 1800s (30m):
127.0.0.1:6379> GETEX sessao:u42 EX
     1800
"jwt_token"

# Verificar novo tempo de expiracao:
127.0.0.1:6379> TTL sessao:u42
(integer) 1799
```

Inspeção de Strings (GETRANGE e STRLEN)

Substrings e Tamanho em Bytes

- `GETRANGE`: Extrai intervalo de caracteres usando offsets de início e fim.
- `STRLEN`: Retorna o tamanho exato da string armazenada em bytes.
- Execução instantânea em memória.

Comandos no redis-cli

```
# Gravar mensagem textual:
127.0.0.1:6379> SET msg "IFCE Campus
     Taua"
OK

# Obter substring pelo intervalo [5,
     10]:
127.0.0.1:6379> GETRANGE msg 5 10
"Campus"

# Medir comprimento em bytes:
127.0.0.1:6379> STRLEN msg
(integer) 16
```

A Importância do Cache, Expiração e Práticas de Atualização

Por que o Cache DEVE ser Expirável (TTL)?

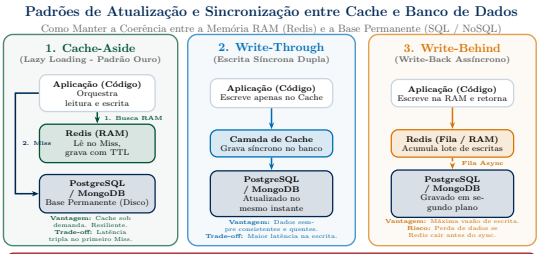
1. A Finitude Física da Memória RAM

- A memória RAM custa significativamente mais por gigabyte que discos NVMe.
- Se gravarmos todas as buscas e consultas no Redis sem expiração, a RAM esgotará rapidamente.
- **Políticas de Evicção:** Quando atinge o `maxmemory`, o Redis descarta chaves via algoritmos LRU (*Least Recently Used*) ou LFU (*Least Frequently Used*).

2. Autocura Contra Dados Obsoletos

- Em arquiteturas distribuídas, sincronizações podem falhar por timeout de rede ou concorrência.
- **Self-Healing:** Um TTL bem calibrado (ex: 60s a 300s) atua como rede de segurança.
- Mesmo que a invalidação ativa falhe, a chave expira e a próxima consulta busca a verdade atualizada na fonte primária.

Padrões de Atualização de Cache e Sincronização



[REGRA CRÍTICA] Atualização no Cache-Aside: Invalidar (`DEL`) em vez de Sobrescrever (`SET`)

Ao atualizar um registro na base permanente, **remova a chave do Redis** (`DEL cache:item:X`).

Nunca tente atualizar o valor no cache na mesma escrita, pois sob concorrência duas threads paralelas podem gravar valores trocados gerando *Race Conditions* incuráveis.

Invalidação no Cache-Aside: O Risco de Race Conditions

O Erro Comum de Iniciante

- Ao atualizar um produto no banco relacional, tentar atualizar o valor diretamente no Redis via `SET`.
- **Cenário de Desastre:** Sob alta concorrência, a Requisição 1 atualiza o SQL e depois a Requisição 2 atualiza o SQL.
- Porém, pela rede, o `SET` da Requisição 1 chega no Redis **depois** do `SET` da Requisição 2!

A Consequência Grave

- O banco relacional permanente fica com o valor novo (da Requisição 2).
- O Redis fica com o valor velho (da Requisição 1) gravado para sempre!
- O cache torna-se inconsistente com a verdade do sistema.

Solução: A Regra de Ouro da Invalidação (DEL)

Invalidação Limpa e Segura

- Ao alterar o banco permanente, **apenas delete a chave do Redis (`DEL`)**.
- Não tente adivinhar o valor do cache na mesma escrita.
- A próxima leitura disparará *Cache Miss*, recarregando a verdade da base SQL com novo TTL.

Fluxo Seguro no Terminal

```
# 1. Atualizar registro no banco SQL:
# UPDATE produtos SET preco=3599
      WHERE id=101;

# 2. Invalidar imediatamente no
      Redis:
127.0.0.1:6379> DEL busca:item:101
(integer) 1

# 3. Proxima leitura busca no SQL e
      repovoa:
127.0.0.1:6379> SET busca:item:101
      "3599" EX 300
OK
```

Políticas de Evicção de Memória sob Sobrecarga

Algoritmos no Limite de RAM

- Ao atingir o limite `maxmemory`:
- **volatile-lru:** Descarta chaves com TTL menos acessadas recentemente (*Recomendado para cache*).
- **allkeys-lru:** Descarta qualquer chave LRU.
- **noeviction:** Retorna erro em comandos de gravação quando a memória enche.

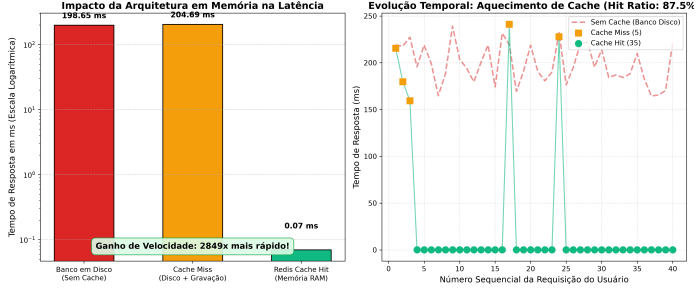
Diretivas no redis.conf

```
# Limite maximo de memoria para o
     Redis:
maxmemory 4gb

# Politica recomendada para camada
     de cache:
maxmemory-policy volatile-lru

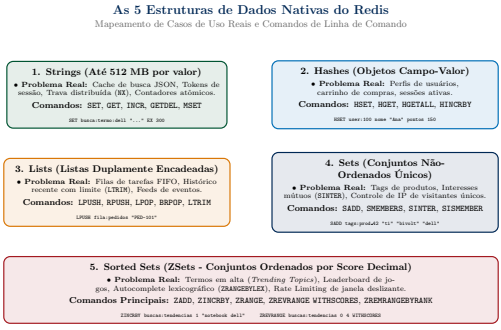
# Inspecionar consumo de memoria no
     CLI:
127.0.0.1:6379> INFO memory
# used_memory_human:1.24M
```

Comprovação Experimental: Benchmarking de Latência



Estruturas Nativas e Modernização com Inteligência Artificial

As 5 Estruturas de Dados Nativas do Redis



Estrutura 2: Hashes (Objetos Campo-Valor)

Modelagem de Entidades

- Mapeia dicionários campo-valor sob uma única chave.
- Permite alterar campos isolados sem trafegar o JSON todo.
- **Casos Reais:** Perfis de usuários, carrinhos e preferências.

Comandos no redis-cli

```
# Criar perfil de aluno com campos:
127.0.0.1:6379> HSET user:aluno:42
     nome "Ana" pontos 80
(integer) 2

# Incrementar pontuacao atomicamente:
127.0.0.1:6379> HINCRBY
     user:aluno:42 pontos 20
(integer) 100

# Ler campo isolado em O(1):
127.0.0.1:6379> HGET user:aluno:42
     pontos
"100"

# Recuperar todos os pares:
127.0.0.1:6379> HGETALL user:aluno:42
1) "nome"    2) "Ana"
3) "pontos"  4) "100"
```

Estrutura 3: Lists (Filas FIFO e Históricos)

Listas Duplamente Encadeadas

- Inserção rápida nas pontas (`LPUSH` / `RPUSH`).
- **Capped Lists:** `LTRIM` mantém tamanho fixo.
- **Casos Reais:** Filas de tarefas FIFO e histórico de buscas recentes.

Comandos no redis-cli

```
# Adicionar buscas recentes:
127.0.0.1:6379> LPUSH hist:42
     "python" "nosql" "redis"
(integer) 3

# Manter apenas as 2 mais recentes:
127.0.0.1:6379> LTRIM hist:42 0 1
OK

# Consultar historico podado:
127.0.0.1:6379> LRANGE hist:42 0 -1
1) "redis"  2) "nosql"

# Consumir da ponta oposta (FIFO):
127.0.0.1:6379> RPOP hist:42
"nosql"
```

Estrutura 4: Sets (Conjuntos Únicos e Álgebra)

Conjuntos Não-Ordenados Únicos

- Garante unicidade de membros em $\mathcal{O}(1)$.
- Suporta álgebra de conjuntos: Interseção (`SINTER`), União (`SUNION`).
- **Casos Reais:** Tags de catálogo e visitantes únicos.

Comandos no redis-cli

```
# Cadastrar tags de dois produtos:
127.0.0.1:6379> SADD tags:prodA "ti"
     "bivolt" "dell"
(integer) 3
127.0.0.1:6379> SADD tags:prodB
     "eletro" "bivolt" "dell"
(integer) 3

# Testar pertinencia em O(1):
127.0.0.1:6379> SISMEMBER tags:prodA
     "dell"
(integer) 1

# Intersecao: Tags compartilhadas:
127.0.0.1:6379> SINTER tags:prodA
     tags:prodB
1) "dell"  2) "bivolt"
```

Estrutura 5: Sorted Sets (Rankings e Tendências)

Conjuntos Ordenados por Pontuação

- Cada membro possui um **score numérico**.
- Mantém a coleção ordenada em memória via SkipList interna.
- **Casos Reais:** *Trending Topics* de busca em tempo real e leaderboards.

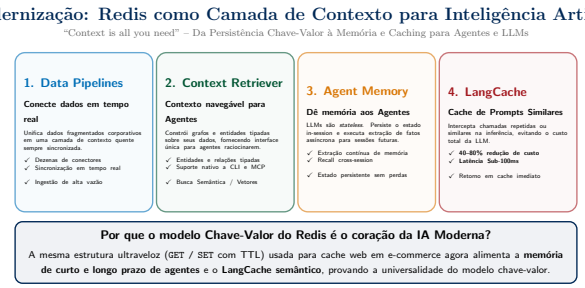
Comandos no redis-cli

```
# Inicializar termos com volume de
    buscas:
127.0.0.1:6379> ZADD rank:buscas 100
    "notebook" 150 "celular"
(integer) 2

# Incrementar busca em tempo real:
127.0.0.1:6379> ZINCRBY rank:buscas
    85 "notebook"
"185"

# Top 2 mais buscados decrescente:
127.0.0.1:6379> ZREVRANGE
    rank:buscas 0 1 WITHSCORES
1) "notebook"  2) "185"
3) "celular"   4) "150"
```

Modernização: Redis como Camada de Contexto para IA



Atividades Práticas e Próximos Passos

Laboratório Hands-On de Hoje

- Executar o script de terminal `redis_cli_guia_pratico.sh`.
- Praticar no terminal `redis-cli` as variações de `SET` (`NX`, `EX`, `KEEPTTL`) e `GET` (`GETDEL`, `MGET`).
- Testar a persistência disparando `BGSAVE` e inspecionando `LASTSAVE`.
- Rodar o script Python interativo `demonstracao_aula04.py` e inspecionar os tempos de latência.

Conexão com a Aula 05

- Na próxima aula (**Aula 05 - 20/10/2026**), aprofundaremos as **Estratégias Avançadas de Caching, Controle de Sessões e Filas de Mensagens (Streams)**.
- Resolver a lista de exercícios práticos da Aula 04 disponível no portal web da disciplina.
- O código e slides estão disponíveis no repositório oficial da turma.

Dúvidas & Discussão

IFCE Campus Tauá – ADS16

reginaldo.fernandes@ifce.edu.br